Global Knowledge®

Expert Reference Series of White Papers

OpenLMI Improves Management in Red Hat Enterprise Linux (RHEL) 7

OpenLMI Improves Management in Red Hat Enterprise Linux (RHEL) 7

Kerry Doyle, MA, ZDNet/CNet.com Associate Editor

Introduction

In its newest release, Red Hat Enterprise Linux (RHEL) 7, Red Hat has created a viable commercial offering for a generally complex operating system. In the past, one challenge for systems administrators has been the steep learning curve for mastering dozens of Linux tools and utilities, each with their own structure and idiosyncrasies. Open Linux Management Infrastructure (OpenLMI) represents an effort by Red Hat to provide unified management of these diverse utilities and systems.

Built on top of existing Linux capabilities, OpenLMI reduces the learning curve and increases the capabilities of both new and experienced IT administrators while supporting new tools and applications. It offers a range of methods to address the needs of system administrators using OpenLMI Commands, modules, and the powerful OpenLMI API.

In this white paper, we'll explore the benefits of OpenLMI as they extend to managing hardware, monitoring remote systems, and configuring services—from complex storage to networks. We also examine the development of LMI and its foundation based on previous management systems. Finally, we assess the OpenLMI architecture and both of its major server- and client-based components.

Managing Tasks in RHEL 7 with OpenLMI

The OpenLMI systems management framework is based on open industry standards. These standards ensure application, platform, and access independence and make future IT development possible largely through not-for-profit working groups and self-directed developers. As a result, OpenLMI goes farther than previous Linux system managers in executing system functions. In the past these standard management agents were used primarily for basic monitoring and reporting.

OpenLMI is built on a model of interactive query, modify, and notify against a single system. The OpenLMI API is standardized, complex, and powerful. As a low-level API, it is adequate for serving the needs of programmers and software engineers, but less useful for the daily management goals of IT administrators.

For that reason the LMIShell, as an integral part of OpenLMI, helps meet the task-oriented needs of administrators who are used to command line interfaces (CLI) and script writing. As a single client application/environment, LMIShell makes it possible to connect with and manage multiple, remote systems and to complete tasks.

In fact, the LMIShell provides helper functions that alleviate the need to revise pre-existing scripts and commands normally found via Internet searches, a common activity on the part of IT administrators. Because the shell is based on Python, and Python-based method calls, working with OpenLMI is greatly simplified.

For example, LMIShell helper functions enable the creation of connection objects that pass commonly used access parameters (i.e., username, password, etc.), effectively eliminating repetitive tasks. The LMIShell CLI is high-level, task-focused, and administrator-friendly and can be used interactively or called from Bash scripts.

On a related front, remote procedural calls (RPC) offer the ability to execute a specified procedure with designated parameters. However, these remote calls can often fail due to network issues. Or else the caller may be unaware as to whether a remote procedure was actually invoked.

As IT administrators manage increasing numbers of application workloads, OpenLMI helps alleviate such call issues. In addition to local and remote access, standard APIs, and scripting interfaces, the framework also provides multiple language bindings. Wrapper functions then enable these data structures to be converted from one language to another.

The OpenLMI Architecture

The OpenLMI framework employs Distributed Management Task Force/Common Information Model (DMTF/CIM) as a baseline reference for server management. DMTF can best be described as a consortium of roughly 160 companies devoted to the development, unification, and implementation of standards that enable effective management of IT environments.

In terms of OpenLMI, the specific framework includes:

1. System management agents (i.e., **LMI Providers**) installed on a managed system
2. **OpenLMI object broker** or *controller* for managing agents and providing an interface to them
3. **Client applications** or scripts to call the management agents through the controller

To understand the OpenLMI framework, it's useful to know that CIM is primarily an object-oriented data model for managed elements. This model makes it possible to have a consistent view of logical and physical objects within a management environment.

To achieve that view, CIM creates "classes" to represent elements, such as hard disk drives, applications, network routers, and related hardware. Then, DMTF/CIM takes the elements relevant to server management and extends them, as required, to support a wide variety of Linux capabilities.

In order to provide administrative functionality, DMTF/CIM object models are then implemented as agents or CIM providers on top of existing Linux tools and utilities. The OpenLMI object broker, also known as Common Information Model Object Manager (CIMOM), manages these agents and provides interfaces, client applications, and scripts for calling the CIM providers to perform tasks.

OpenLMI not only enables the configuring, managing, and monitoring of complex storage and networks via system management agents, it also provides IT administrators with call system management functions using accessible, commonly used languages. All of these interfaces are implemented as language bindings to the underlying system agents.

In addition to building management agents, which capitalizes on pre-existing Linux functions, OpenLMI simplifies the managing of bare metal production servers. In general, the issue with bare metal provisioning is often due to its complex, time-consuming initializing processes.

Previously, managing Linux systems usually involved logging into a server using ssh and then running the dozens of tools and utilities as needed. As an alternative, the previously mentioned LMIShell application provides an easy-to-use interface that complements the OpenLMI system management agents. It's a task-oriented, extensible CLI specifically designed for system administrators and considerably easier to use than the low-level OpenLMI API.

In terms of providing true extensibility, the system management agents, otherwise known as LMI providers, offer a diverse and growing number of capabilities and features. Currently, there are providers for networking, managing users/groups, and software. For storage and mounting, the provider relies on the former Anaconda library for storage management.

Background of OpenLMI

Red Hat had previously pursued development on a unified management framework in earlier versions of RHEL. Most notable was Matahari, which represented a cross-platform collection of APIs that provided agents for systems management and monitoring.

Based on Apache Qpid Management Framework (QMF), it enabled the building of remote APIs on top of AMQP, an open protocol for messaging. The Matahari agent was an application that provided access to features in the Matahari library. Based on Host and Services functionality, it enabled users to actively monitor and control services on remote hosts.

As represented by the current OpenLMI framework, such capabilities are critical to enterprise use of RHEL as well as for providing High Availability (HA) in cloud-based environments. For example, the type of Host controls found in Matahari enabled remote shutdowns, reboots, and retrieval of basic host information (i.e., CPU, RAM, load averages, etc.) Service agents enabled status monitoring and system controls.

Another goal for the Matahari project was to provide QMF code to third parties for creating customized agents. However, further development on the framework was deprecated starting with the RHEL 6.3 release.

The comprehensive retooling that resulted in the current RHEL 7 release and the OpenLMI framework demonstrates the importance of streamlining the management of configuration, administration, services, and security. Moreover, OpenLMI not only reduces the learning curve for new and mid-level system administrators by hiding the complexity of the underlying Linux system, it also increases the productivity of experienced IT leaders.

As a result, administrators can access a straightforward, consistent, yet powerful CLI and API for performing a range of tasks. The goal of OpenLMI is to offer a comprehensive set of tools to configure, manage, and monitor remote servers. But it's also helping to make RHEL 7 one of the most advanced and accessible IT platforms available.

OpenLMI's Diverse Functionality

It's useful to compare the functionality of OpenLMI to other system management tools. As mentioned previously, OpenLMI is built on a model of interactive query, modify, and notify against a single system via standardized APIs. It enables administrators, by extension, to remotely add or remove a service, determine its state (i.e., running, stopped, failed, etc.), and enable or start a new service.

Such capabilities extend to providing client applications, APIs with task-oriented scripting and GUIs. In contrast to similar systems management tools such as Puppet, OpenLMI offers intensive, configuration-specific features.

For example when using Puppet, IT administrators can perform multiple iterations of certain tasks. Since Puppet is idempotent, it enables technicians to run multiple instances of the same configuration and achieve the same result each time.

These abilities can be critical when setting up multiple systems with identical configurations. Modified Puppet recipes enable IT administrators to run unlimited propagations of the same configuration. On the other hand, if each server had a unique configuration, OpenLMI would offer the most optimal tools for setting up those instances.

In many ways, system administrators could access the complementary strengths of each tool by utilizing Puppet and OpenLMI together. For example, it would be possible to create Puppet agents that can call OpenLMI agents to perform certain configuration tasks. Such capabilities offer true diversity and efficiency.

Conclusion

Included as an integral part of RHEL 7, OpenLMI offers significant gains in managing the wealth of available Linux tools. The fact that the interface can be accessed via multiple languages, including C/C++, Python, Java, and a CLI further enhances its versatility.

In addition, the ability to use OpenLMI on either remote physical servers or Virtual Machine (VM) guests represents a critical step forward for RHEL 7. In the past, configuring and managing bare metal production servers as well as VM guests was both cumbersome and time consuming. It also required advanced technical skill on the part of IT administrators. Now, improved accessibility, multiple language bindings, standardized APIs, and standard scripting interfaces significantly broaden the management potential and promise future adoption for RHEL 7 across the enterprise.

The fact remains that IT management structures continue to radically change as increasing numbers of alternatives to Windows-based environments are brought to market. As RHEL continues to grow in enterprise adoption, the option of maintaining Windows and Linux systems within the same environment will become ever more compelling.

Learn More

Learn more about how you can improve productivity, enhance efficiency, and sharpen your competitive edge through training.

- Red Hat® System Administration I (RH124)
- Red Hat® System Administration II (RH134)
- Red Hat® System Administration III (RH254)

Visit **www.globalknowledge.com** or call **1-800-COURSES (1-800-268-7737)** to speak with a Global Knowledge training advisor.

About the Author

Kerry Doyle (MA, MSr, CPL) writes for a diverse group of companies based in technology, business, and higher education. As an educator and former editor at PC/Computing, reporter for PCWeek magazine, and associate editor at www.ZDNet.com, he has written extensively on high-tech issues for over 15 years. He specializes in computing trends vital to SMBs and enterprises alike—from virtualization and cloud computing to disaster recovery and network storage.